Name: Vansh Jain
Roll No: 124A8036
Class/Div: SE G

Experiment No.: 04

Title: Process Management: Linux Process Creation and Control

Objective: Linux Process

A. **Write a program in Linux to create a child process using the fork() system call and display the Process ID (PID) of both the parent and child processes using getpid() and getppid().**

B. **Write a program to demonstrate process synchronization and termination using the wait() and waitpid() system calls.**

Theory:
Linux Process Management
A process in Linux is an instance of a running program. Each process is identified by a unique Process ID (PID). Linux follows a parent–child process hierarchy, where new processes are created by existing ones.

fork() System Call
The fork() system call is used to create a new process.

- The original process is called the **parent process**
- The newly created process is called the **child process**
- fork() returns:
  - 0 to the child process
  - **PID of the child** to the parent process

After fork(), both processes execute concurrently.

getpid()

- Returns the **PID of the current process**
- Used to identify whether the code is executing in the parent or child process

getppid()

- Returns the **PID of the parent process**
- Useful to verify the parent–child relationship between processes

wait() System Call

- Makes the **parent process wait** until any one of its child processes finishes execution
- Prevents the creation of **zombie processes**
- Ensures proper synchronization between parent and child

waitpid() System Call

- Allows the parent process to wait for a **specific child process**
- Provides more control over process termination
- Can be used for blocking or non-blocking waits

Output/Result:

```
siesgst@siesgst-OptiPlex-3000:/home/samiii1214$ cd ..
siesgst@siesgst-OptiPlex-3000:/home$ sudo mkdir exp4
[sudo] password for siesgst:
siesgst@siesgst-OptiPlex-3000:/home$ ls
alok  cari  catfish  catish  exp4  hadi  nishank  oslabb1  oslabb3  samiii1214  siesgst
siesgst@siesgst-OptiPlex-3000:/home$ cd exp4
siesgst@siesgst-OptiPlex-3000:/home/exp4$ sudo nano non_preemptive_scheduling.c
siesgst@siesgst-OptiPlex-3000:/home/exp4$ sudo gcc non_preemptive_scheduling.c -o non_preemptive_scheduling
siesgst@siesgst-OptiPlex-3000:/home/exp4$ ./non_preemptive_scheduling
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 3
Enter Arrival Time and Burst Time for P2: 0 3
Enter Arrival Time and Burst Time for P3: 0 3

PID     BT      WT      TAT
1       3       0       3
2       3       3       6
3       3       6       9

Average Waiting Time = 3.00
Average Turnaround Time = 6.00
siesgst@siesgst-OptiPlex-3000:/home/exp4$
```

```
Activities   Terminal   Feb 6 12:30
siesgst@siesgst-OptiPlex-3000: /home/exp4
GNU nano 4.8                    non_preemptive_scheduling.c
#include <stdio.h>

typedef struct {
    int pid, at, bt, wt, tat;
} Process;

int main() {
    int n, i, j;
    Process p[10], temp;
    float avgWT = 0, avgTAT = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("Enter Arrival Time and Burst Time for P%d: ", p[i].pid);
        scanf("%d %d", &p[i].at, &p[i].bt);
    }

    // SJF (Non-preemptive): sort by burst time
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (p[i].bt > p[j].bt) {
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }

    p[0].wt = 0;
    for (i = 1; i < n; i++) {
        p[i].wt = p[i - 1].wt + p[i - 1].bt;
    }

    for (i = 0; i < n; i++) {
        p[i].tat = p[i].wt + p[i].bt;
        avgWT += p[i].wt;
        avgTAT += p[i].tat;
    }

    printf("\nPID\tBT\tWT\tTAT\n");
    for (i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\n", p[i].pid, p[i].bt, p[i].wt, p[i].tat);
    }

    printf("\nAverage Waiting Time = %.2f", avgWT / n);
    printf("\nAverage Turnaround Time = %.2f\n", avgTAT / n);

    return 0;
[ Read 52 lines ]
^G Get Help   ^O Write Out   ^W Where Is   ^K Cut Text   ^J Justify   ^C Cur Pos   M-U Undo   ^^ Mark Text   ^] To Bracket   ^Y Previous   ^B Back   ^← Prev Word
^X Exit       ^R Read File   ^\ Replace    ^U Paste Text ^T To Spell   ^_ Go To Line M-E Redo   M-6 Copy Text   ^/ Where Was   ^V Next     ^F Forward   ^→ Next Word
```

```
siesgst@siesgst-OptiPlex-3000:/home/exp4$ sudo nano non_preemptive_scheduling.c
siesgst@siesgst-OptiPlex-3000:/home/exp4$
siesgst@siesgst-OptiPlex-3000:/home/exp4$ sudo nano preemptive_scheduling.c
siesgst@siesgst-OptiPlex-3000:/home/exp4$ sudo gcc preemptive_scheduling.c -o preemptive_scheduling
siesgst@siesgst-OptiPlex-3000:/home/exp4$ ./preemptive_scheduling
Enter number of processes: 4
Enter Arrival Time and Burst Time for P1: 0 4
Enter Arrival Time and Burst Time for P2: 0 4
Enter Arrival Time and Burst Time for P3: 0 4
Enter Arrival Time and Burst Time for P4: 0 4

PID     BT      WT      TAT
1       4       0       4
2       4       4       8
3       4       8       12
4       4       12      16
siesgst@siesgst-OptiPlex-3000:/home/exp4$
```

```
GNU nano 4.8                    preemptive_scheduling.c
#include <stdio.h>
#include <limits.h>

int main() {
    int n, i, time = 0, completed = 0;
    int bt[10], rt[10], at[10], wt[10], tat[10];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("Enter Arrival Time and Burst Time for P%d: ", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
        rt[i] = bt[i];
    }

    int shortest, finishTime;
    int minRT = INT_MAX;

    while (completed != n) {
        shortest = -1;
        minRT = INT_MAX;

        for (i = 0; i < n; i++) {
            if (at[i] <= time && rt[i] > 0 && rt[i] < minRT) {
                minRT = rt[i];
                shortest = i;
            }
        }

        if (shortest == -1) {
            time++;
            continue;
        }

        rt[shortest]--;
        time++;

        if (rt[shortest] == 0) {
            completed++;
            finishTime = time;
            tat[shortest] = finishTime - at[shortest];
            wt[shortest] = tat[shortest] - bt[shortest];
        }
    }

    printf("\nPID\tBT\tWT\tTAT\n");
    for (i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);
    }
                                    [ Read 53 lines ]
^G Get Help   ^O Write Out   ^W Where Is   ^K Cut Text   ^J Justify   ^C Cur Pos   M-U Undo   M-A Mark Text   M-] To Bracket   M-Q Previous   ^B Back   ^← Prev Word
^X Exit       ^R Read File   ^\ Replace    ^U Paste Text ^T To Spell   ^_ Go To Line M-E Redo   M-6 Copy Text   ^Q Where Was   M-W Next   ^F Forward   ^→ Next Word
```

```
GNU nano 4.8                    round_robin_scheduling.c                    Modified
#include <stdio.h>

int main() {
    int n, i, time = 0, remain, flag = 0, tq;
    int wt = 0, tat = 0;
    int at[10], bt[10], rt[10];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("Enter Arrival Time and Burst Time for P%d: ", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
        rt[i] = bt[i];
    }

    printf("Enter Time Quantum: ");
    scanf("%d", &tq);

    remain = n;
    int done;
    int waiting[10] = {0}, turnaround[10] = {0};

    while (remain != 0) {
        done = 1;
        for (i = 0; i < n; i++) {
            if (rt[i] > 0 && at[i] <= time) {
                done = 0;
                if (rt[i] > tq) {
                    time += tq;
                    rt[i] -= tq;
                } else {
                    time += rt[i];
                    waiting[i] = time - at[i] - bt[i];
                    rt[i] = 0;
                    remain--;
                }
            }
        }
        if (done) time++; // If no process is ready, increment time
    }

    printf("\nPID\tAT\tBT\tWT\tTAT\n");
    for (i = 0; i < n; i++) {
        turnaround[i] = bt[i] + waiting[i];
        printf("%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], waiting[i], turnaround[i]);
        wt += waiting[i];
        tat += turnaround[i];
    }

    printf("\nAverage Waiting Time = %.2f\n", (float)wt / n);
^G Get Help   ^O Write Out   ^W Where Is   ^K Cut Text   ^J Justify   ^C Cur Pos   M-U Undo   M-A Mark Text   M-] To Bracket   M-Q Previous   ^B Back   ^← Prev Word
^X Exit       ^R Read File   ^\ Replace    ^U Paste Text ^T To Spell   ^_ Go To Line M-E Redo   M-6 Copy Text   ^Q Where Was   M-W Next   ^F Forward   ^→ Next Word
```

```
siesgst@siesgst-OptiPlex-3000:/home/exp4$ sudo gcc round_robin_scheduling.c -o round_robin_scheduling
siesgst@siesgst-OptiPlex-3000:/home/exp4$ ./round_robin_scheduling
Enter number of processes: 2
Enter Arrival Time and Burst Time for P1: 0 2
Enter Arrival Time and Burst Time for P2: 0 2
Enter Time Quantum: 2

PID     AT      BT      WT      TAT
1       0       2       0       2
2       0       2       2       4

Average Waiting Time = 1.00
Average Turnaround Time = 3.00
siesgst@siesgst-OptiPlex-3000:/home/exp4$
```

```
Activities   Terminal   Feb 6 12:45
siesgst@siesgst-OptiPlex-3000: /home/exp4
GNU nano 4.8                    priority_scheduling.c                    Modified
#include <stdio.h>

typedef struct {
    int pid, at, bt, priority, wt, tat;
} Process;

int main() {
    int n, i, j;
    Process p[10], temp;
    float avgWT = 0, avgTAT = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("Enter Arrival Time, Burst Time and Priority for P%d: ", p[i].pid);
        scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].priority);
    }

    // Sort by priority (lower number = higher priority)
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (p[i].priority > p[j].priority) {
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }

    p[0].wt = 0;
    for (i = 1; i < n; i++) {
        p[i].wt = p[i - 1].wt + p[i - 1].bt;
    }

    for (i = 0; i < n; i++) {
        p[i].tat = p[i].wt + p[i].bt;
        avgWT += p[i].wt;
        avgTAT += p[i].tat;
    }

    printf("\nPID\tPriority\tBT\tWT\tTAT\n");
    for (i = 0; i < n; i++) {
        printf("%d\t%d\t\t%d\t%d\t%d\n", p[i].pid, p[i].priority, p[i].bt, p[i].wt, p[i].tat);
    }

    printf("\nAverage Waiting Time = %.2f", avgWT / n);
    printf("\nAverage Turnaround Time = %.2f\n", avgTAT / n);

    return 0;
^G Get Help   ^O Write Out   ^W Where Is   ^K Cut Text   ^J Justify   ^C Cur Pos   M-U Undo   M-A Mark Text   M-] To Bracket   M-Q Previous   ^B Back   ^← Prev Word
^X Exit       ^R Read File   ^\ Replace    ^U Paste Text ^T To Spell   ^_ Go To Line M-E Redo   M-6 Copy Text   ^Q Where Was   M-W Next   ^F Forward   ^→ Next Word
```

```
siesgst@siesgst-OptiPlex-3000:/home/exp4$ sudo gcc priority_scheduling.c -o priority_scheduling
siesgst@siesgst-OptiPlex-3000:/home/exp4$ ./priority_scheduling
Enter number of processes: 2
Enter Arrival Time, Burst Time and Priority for P1: 0 5 2
Enter Arrival Time, Burst Time and Priority for P2: 0 5 2

PID     Priority        BT      WT      TAT
1       2               5       0       5
2       2               5       5       10

Average Waiting Time = 2.50
Average Turnaround Time = 7.50
siesgst@siesgst-OptiPlex-3000:/home/exp4$
```

Conclusion: In this experiment, Linux process creation and control were successfully demonstrated using system calls. The fork() system call was used to create a child process, while getpid() and getppid() helped identify parent and child processes. The use of wait() and waitpid() ensured proper synchronization and controlled termination of processes. These concepts are fundamental to understanding Linux process management.